

**MAHARASHTRA STATE BOARD OF TECHNICAL EDUCATION,
MUMBAI**

**PROJECT REPORT ON
FITBAT - Fitness Battles & AI Health Monitor**

SUBMITTED BY:

1. Kshitij Pawar (Roll No. 25942)
2. Swaraj Kadam (Roll No. 25928)

GUIDED BY:

Prof. Neha S.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

Fr. Agnel Polytechnic, Vashi, Navi Mumbai - 400703

ACADEMIC YEAR 2026-2027

FR. AGNEL POLYTECHNIC VASHI, NAVI MUMBAI - 400703

PROJECT NAME:

FITBAT - Fitness Battles & AI Health Monitor

NAME	ROLL NO
1. Kshitij Pawar	25942
2. Swaraj Kadam	25928

Prof. Neha S.
(SUBJECT TEACHER)

Prof. Neha S.
(PROJECT GUIDE)

Contents

ACKNOWLEDGEMENT	4
FITBAT - Fitness Battles & AI Health Monitor Using Python	5
1. Introduction	5
2. Problem Statement	6
3. Objectives	7
4. Technologies Used	8
5. Hardware Requirements	9
6. Software Requirements	10
7. Working of the System	11
8. System Flow	12
9. CODE (Computer Vision Pose Estimation Engine)	13
10. Important Functions Used	16
11. Output & User Interface	17
12. Advantages	18
13. Limitations	19
14. Future Scope	20
15. Conclusion	21

ACKNOWLEDGEMENT

We express our profound gratitude to our project guide, **Prof. Neha S.**, for her valuable guidance, constant encouragement, and constructive suggestions throughout the course of this project. Her expertise in Artificial Intelligence and Machine Learning proved invaluable in designing the real-time biometric pose estimation pipelines and architecture.

We are also thankful to the **Department of Artificial Intelligence & Machine Learning** and **Fr. Agnel Polytechnic, Vashi** for providing us with state-of-the-art laboratory infrastructure, computational facilities, and continuous support to accomplish this project.

Finally, we extend our heartfelt appreciation to our parents, faculty members, and peers whose encouragement and cooperation enabled us to successfully bring the **FITBAT** project to fruition.

Kshitij Pawar (25942)
Swaraj Kadam (25928)
Department of AI & ML

FITBAT - Fitness Battles & AI Health Monitor Using Python

1. Introduction

Physical inactivity and improper exercise postures are among the primary causes of musculoskeletal injuries and fitness stagnation globally. While traditional mobile fitness apps offer workout logging and video tutorials, they lack real-time computer vision biomechanical feedback, interactive multiplayer competitive incentives, and unified intelligent dietary coaching.

FITBAT is an end-to-end, intelligent fitness battle platform and AI health monitor engineered with **Python, WebRTC, and Computer Vision**. By leveraging 3D landmark coordinate tracking via MediaPipe Pose, FITBAT transforms any standard webcam or smartphone camera into an automated, high-accuracy personal fitness referee. It provides real-time biomechanical repetition counting, cheat-prevention posture validation, low-latency peer-to-peer multiplayer fitness duels over WebSockets/WebRTC, automated GPS route breadcrumb mapping via OpenStreetMap, and an AI-powered conversational nutrition and hydration coach.

Key Innovative Highlights:

- **Computer Vision Pose Ref:** Biometric angle calculation across 13 core disciplines with anti-cheat validation.
- **Arcade Fitness Battles:** Real-time peer video streaming (WebRTC) and instant matchmaking duels.
- **AI Health Coach:** Conversational diet tracking with calorie, macro (P/C/F), and water hydration logging.
- **Open-Source GPS Tracker:** Multi-sport route mapping without third-party API keys or subscription fees.

2. Problem Statement

Traditional fitness tracking applications suffer from fundamental limitations that hinder user progress and engagement:

- 1. Absence of Real-Time Form Correction:** Exercising with incorrect biomechanics (such as sagging hips during pushups or shallow knee flexion during squats) leads to acute tendon strain and long-term joint injury. Standard video tutorials cannot provide corrective feedback.
- 2. Lack of Social & Competitive Motivation:** Solo workouts suffer from high attrition rates. Existing apps rely on passive step counters rather than exciting, head-to-head competitive multiplayer gamification.
- 3. Manual & Tedious Calorie Logging:** Traditional calorie counting requires manual barcode scanning and tedious gram lookups. Users frequently abandon tracking due to complexity.
- 4. Expensive Hardware & Proprietary Map Paywalls:** Many wearable fitness ecosystems require expensive proprietary smartwatches and map APIs that mandate billing keys, restricting access for young athletes and students.

Therefore, there is an urgent demand for an integrated, hardware-free fitness ecosystem that combines computer vision biomechanics, real-time peer battles, and automated AI nutrition assistance.

3. Objectives

- 1. Develop Real-Time Computer Vision Exercise Tracking:** Implement mathematical joint angle analysis and spatial displacement algorithms to accurately count repetitions and assess posture across exercises (Pushups, Squats, Frog Jumps, Bicep Curls, etc.).
- 2. Build an Anti-Cheat Biomechanical Verification System:** Ensure users cannot cheat repetitions through shallow movement, seated posture, or camera nodding by applying horizontal torso and vertical spine validity checks.
- 3. Architect Low-Latency Peer-to-Peer Fitness Arena:** Enable real-time 1v1 fitness battles using WebRTC peer camera feeds and WebSocket synchronization for instant rep bursts, combo surges, and live scorecards.
- 4. Create AI-Powered Conversational Health Monitor:** Provide automated natural language meal calorie estimation, macronutrient breakdown (protein, carbs, fats), and hydration logging.
- 5. Deploy Hardware-Free Multi-Sport GPS Tracking:** Deliver outdoor cycling, running, and walking breadcrumb mapping with kilometer splits using 100% open-source OpenStreetMap layers without third-party API billing restrictions.
- 6. Gamify Fitness with Live Dynamic Leaderboards:** Maintain instant global and discipline-specific leaderboard rankings to incentivize continuous athletic progression.

4. Technologies Used

Component / Layer	Technology / Framework	Role in FITBAT Project
Programming Language	Python 3.10+ / ES6 JavaScript	Backend server logic, REST APIs, WebSockets, client-side CV processing.
Backend Framework	FastAPI & Uvicorn ASGI	High-performance asynchronous REST endpoints, JWT auth, and WebSocket management.
Computer Vision Engine	MediaPipe Pose & Trigonometry	33 3D skeletal landmark detection, vector angle calculations, and rep classification.
Real-Time Communications	WebRTC & WebSockets	Peer-to-peer video streaming and millisecond-level rep data synchronization.
Database Layer	SQLite3 Relational DB	Persistent storage of user profiles, nutrition logs, exercise statistics, and match history.
AI Conversational Agent	OpenRouter API / Local Calorie DB	Intelligent health recommendations, meal decomposition, and coaching advice.
GPS & Map Engine	Leaflet.js & OpenStreetMap	Interactive live outdoor GPS route tracking without API key requirements.

5. Hardware Requirements

Hardware Component	Minimum Specification	Recommended Specification
Processor (CPU)	Dual Core 2.0 GHz Intel / AMD	Intel Core i5 / AMD Ryzen 5 or Apple Silicon M-series
Random Access Memory	4 GB RAM	8 GB - 16 GB DDR4/DDR5 RAM
Camera / Sensor	Standard 720p HD Webcam (30 FPS)	1080p FHD Webcam (60 FPS) with wide-angle FOV
Storage	2 GB available SSD/HDD storage	5 GB high-speed NVMe SSD
Network Connection	1 Mbps broadband internet	10+ Mbps stable broadband / 4G / 5G connection
Display Output	1280 x 720 resolution monitor	1920 x 1080 FHD IPS display

6. Software Requirements

Software Environment	Specification / Version
Operating System	Windows 10 / 11, Linux (Ubuntu 20.04+), macOS Monterey+
Python Environment	Python 3.10, 3.11, or 3.12 (64-bit standard runtime)
Web Browser	Google Chrome 95+, Microsoft Edge 95+, Mozilla Firefox 90+
Web Server / ASGI	Uvicorn ASGI Server with FastAPI Framework
Client-Side Libraries	MediaPipe Pose JS, Leaflet.js, Web Audio API
Database Management	SQLite3 (Embedded, zero-configuration database engine)
Code Editor / IDE	Visual Studio Code, PyCharm, or Antigravity AI IDE

7. Working of the System

- 1. Video Acquisition & Skeletal Landmark Inference:** The user grants camera access via HTML5 `getUserMedia`. The video frames are processed at 30 FPS through the MediaPipe Pose neural pipeline, extracting 33 key landmark coordinates in normalized (X, Y, Z) space.
- 2. Joint Angle & Posture Vector Math:** The system computes 2D and 3D spatial vectors formed by adjacent limbs (e.g., Shoulder-Elbow-Wrist for arm exercises, Hip-Knee-Ankle for leg exercises) using the arc-tangent trigonometric formulation: $\theta = |\arctan2(C_y - B_y, C_x - B_x) - \arctan2(A_y - B_y, A_x - B_x)|$.
- 3. Anti-Cheat Posture Verification:** Before allowing repetitions to increment, the system performs geometric validity checks. For instance, pushups require a horizontal torso ($|\text{Shoulder}_y - \text{Hip}_y| < 0.16$) and reject seated or standing postures.
- 4. State Machine Repetition Transition:** A finite state machine (e.g., IDLE $\rightarrow$ DOWN $\rightarrow$ UP $\rightarrow$ REP_COUNTED) monitors threshold crossings. Hysteresis thresholds prevent false double counts.
- 5. Real-Time Multiplayer Networking:** When entering the Battle Arena, WebSocket connections assign users into match rooms. WebRTC peer connections establish direct media streams for full-duplex video display.
- 6. GPS Tracking & Health Logging:** Outdoor activities record GPS breadcrumbs on an OpenStreetMap Leaflet layer. The conversational health monitor logs meals and computes calories, macronutrients, and hydration targets.

8. System Flow

[USER CAMERA FEED]



[MEDIAPIPE POSE ESTIMATION] (Extracts 33 Landmark Points: (X, Y, Z))



[BIOMETRIC ANGLE & VECTOR CALCULATION] (Joint Angle & Posture Math)



[ANTI-CHEAT POSTURE VALIDATION] (Rejects improper body stances)



[FINITE STATE MACHINE (FSM)] (Checks Hysteresis: Inflexion & Lockout)



[REP REGISTERED & HUD BURST TRIGGERED]



[WEBSOCKET / WEBRTC PACKET TRANSMISSION] (Opponent Receives Score & Video)



[SQLITE DATABASE COMMIT] (XP, Leaderboard Rank, Match History Updated)

9. CODE (Computer Vision Pose Engine in Python)

The following Python module represents the core Computer Vision Biometric Repetition Counting and Anti-Cheat Posture Verification Engine utilized in FITBAT:

```

"""
FITBAT - Real-Time AI Computer Vision Pose Tracking & Biometric Rep Engine
Department of Artificial Intelligence & Machine Learning
Project: FITBAT - Fitness Battles & AI Health Monitor
Authors: Kshitij Pawar (Roll: 25942), Swaraj Kadam (Roll: 25928)
Guide: Prof. Neha S.
"""

import math
import time
from typing import Dict, Tuple, List, Optional

class Landmark:
    def __init__(self, x: float, y: float, z: float = 0.0, visibility: float = 1.0):
        self.x = x
        self.y = y
        self.z = z
        self.visibility = visibility

class FitbatPoseCvEngine:
    """
    Core Computer Vision Pose Estimation & Biometric Motion Analysis Engine.
    Implements 3D skeletal joint trigonometric calculations, form posture
    verification, and anti-cheat biomechanical threshold validation.
    """

    # Standard MediaPipe Joint Indices
    NOSE = 0
    LEFT_SHOULDER = 11
    RIGHT_SHOULDER = 12
    LEFT_ELBOW = 13
    RIGHT_ELBOW = 14
    LEFT_WRIST = 15
    RIGHT_WRIST = 16
    LEFT_HIP = 23
    RIGHT_HIP = 24
    LEFT_KNEE = 25
    RIGHT_KNEE = 26
    LEFT_ANKLE = 27
    RIGHT_ANKLE = 28

    def __init__(self, exercise_id: str = "pushups"):
        self.exercise_id = exercise_id
        self.rep_count = 0
        self.stage = "idle"
        self.form_score = 1.0
        self.form_feedback = "Position yourself in camera frame"
        self.last_rep_timestamp = 0.0
        self.stable_frames = 0
        self.baseline_hip_y = 0.0
        self.deepest_crouch_y = 0.0

    @staticmethod
    def calculate_angle(a: Landmark, b: Landmark, c: Landmark) -> float:
        """
        Calculates the interior 2D/3D joint angle formed by three spatial coordinates:
        Vertex point 'b' connected to endpoints 'a' and 'c'.
        Returns angle in degrees [0.0 - 180.0].

```

9. CODE (Continued - Part 2)

```

"""
if not a or not b or not c:
    return 180.0

radians = math.atan2(c.y - b.y, c.x - b.x) - math.atan2(a.y - b.y, a.x - b.x)
angle = abs(math.degrees(radians))
if angle > 180.0:
    angle = 360.0 - angle
return round(angle, 1)

@staticmethod
def calculate_distance(a: Landmark, b: Landmark) -> float:
    """Euclidean distance between two landmarks in normalized viewport coordinates."""
    if not a or not b:
        return 0.0
    return math.sqrt((a.x - b.x) ** 2 + (a.y - b.y) ** 2)

def process_frame_landmarks(self, landmarks: Dict[int, Landmark]) -> Tuple[int, str, float]:
    """
    Processes normalized skeletal landmarks, performs biometric checks,
    and updates rep count and live biomechanical form feedback.
    """
    ls = landmarks.get(self.LEFT_SHOULDER)
    rs = landmarks.get(self.RIGHT_SHOULDER)
    le = landmarks.get(self.LEFT_ELBOW)
    re = landmarks.get(self.RIGHT_ELBOW)
    lw = landmarks.get(self.LEFT_WRIST)
    rw = landmarks.get(self.RIGHT_WRIST)
    lh = landmarks.get(self.LEFT_HIP)
    rh = landmarks.get(self.RIGHT_HIP)
    lk = landmarks.get(self.LEFT_KNEE)
    rk = landmarks.get(self.RIGHT_KNEE)
    la = landmarks.get(self.LEFT_ANKLE)
    ra = landmarks.get(self.RIGHT_ANKLE)

    # Joint Angles
    l_arm_angle = self.calculate_angle(ls, le, lw)
    r_arm_angle = self.calculate_angle(rs, re, rw)
    l_leg_angle = self.calculate_angle(lh, lk, la)
    r_leg_angle = self.calculate_angle(rh, rk, ra)

    mid_shoulder_y = (ls.y + rs.y) / 2.0 if (ls and rs) else 0.0
    mid_hip_y = (lh.y + rh.y) / 2.0 if (lh and rh) else 0.0
    mid_shoulder_x = (ls.x + rs.x) / 2.0 if (ls and rs) else 0.0
    mid_hip_x = (lh.x + rh.x) / 2.0 if (lh and rh) else 0.0
    current_time = time.time()

    # -----
    # 1. PUSHUPS: Prone Horizontal Plank + 90 Degree Elbow Lockout
    # -----
    if self.exercise_id == "pushups":
        vert_dist = mid_hip_y - mid_shoulder_y
        horiz_dist = abs(mid_hip_x - mid_shoulder_x)

        if vert_dist > 0.18 and horiz_dist < 0.20:
            self.stage = "idle"
            self.form_feedback = "Not pushup stance! Lie horizontally on floor."
            return self.rep_count, self.form_feedback, self.form_score

```

9. CODE (Continued - Part 3)

```

        elbow_angle = min(l_arm_angle, r_arm_angle)

        if elbow_angle <= 95:
            if self.stage != "down":
                self.stage = "down"
                self.stable_frames = 0
                self.form_feedback = "Excellent depth! Push all the way up."
                self.stable_frames += 1

        elif elbow_angle >= 150 and self.stage == "down" and self.stable_frames >= 2:
            if (current_time - self.last_rep_timestamp) > 0.6:
                self.rep_count += 1
                self.last_rep_timestamp = current_time
                self.stage = "up"
                self.stable_frames = 0
                self.form_score = min(1.0, 0.85 + (0.15 * (180 - elbow_angle) / 30.0))
                self.form_feedback = f"Rep {self.rep_count} Counted! Perfect Pushup."

# -----
# 2. BODYWEIGHT SQUATS: Parallel Femur Dip (Knee <= 104 deg)
# -----
elif self.exercise_id == "squats":
    knee_angle = min(l_leg_angle, r_leg_angle)

    if knee_angle <= 104:
        if self.stage != "down":
            self.stage = "down"
            self.stable_frames = 0
            self.form_feedback = "Deep parallel squat! Drive through heels."
            self.stable_frames += 1

    elif knee_angle >= 158 and self.stage == "down" and self.stable_frames >= 2:
        if (current_time - self.last_rep_timestamp) > 0.6:
            self.rep_count += 1
            self.last_rep_timestamp = current_time
            self.stage = "up"
            self.stable_frames = 0
            self.form_feedback = f"Rep {self.rep_count} Counted! Power Squat."

# -----
# 3. FROG JUMPS: Plyometric Deep Crouch to Explosive Jump
# -----
elif self.exercise_id == "frog_jumps":
    knee_angle = min(l_leg_angle, r_leg_angle)
    if not self.baseline_hip_y:
        self.baseline_hip_y = mid_hip_y

    if knee_angle <= 98 and (mid_hip_y >= self.baseline_hip_y + 0.04):
        if self.stage != "crouch":
            self.stage = "crouch"
            self.deepest_crouch_y = mid_hip_y
            self.stable_frames = 0
            self.form_feedback = "Deep crouch! Explode into jump!"
        if mid_hip_y > self.deepest_crouch_y:
            self.deepest_crouch_y = mid_hip_y
        self.stable_frames += 1

```

9. CODE (Continued - Part 4)

```
        elif knee_angle >= 152 and (self.deepest_crouch_y - mid_hip_y >= 0.07):
            if self.stage == "crouch" and self.stable_frames >= 2:
                if (current_time - self.last_rep_timestamp) > 0.55:
                    self.rep_count += 1
                    self.last_rep_timestamp = current_time
                    self.stage = "landed"
                    self.stable_frames = 0
                    self.form_feedback = f"Rep {self.rep_count} Counted! Explosive Leap."

# -----
# 4. BICEP CURLS: Arm Flexion (Elbow <= 50 deg to >= 155 deg)
# -----
elif self.exercise_id == "bicep_curls":
    elbow_angle = min(l_arm_angle, r_arm_angle)
    if elbow_angle <= 50:
        if self.stage != "curled":
            self.stage = "curled"
            self.stable_frames = 0
            self.form_feedback = "Peak contraction! Lower dumbbells slowly."
            self.stable_frames += 1
        elif elbow_angle >= 155 and self.stage == "curled" and self.stable_frames >= 2:
            if (current_time - self.last_rep_timestamp) > 0.5:
                self.rep_count += 1
                self.last_rep_timestamp = current_time
                self.stage = "extended"
                self.stable_frames = 0
                self.form_feedback = f"Rep {self.rep_count} Counted! Full Extension."

return self.rep_count, self.form_feedback, self.form_score
```


10. Important Functions Used

Function / Method	Module / Class	Technical Description
calculate_angle(a, b, c)	FitbatPoseCVEngine	Computes normalized interior angle using 2D arctangent difference across three landmarks.
process_frame_landmarks()	FitbatPoseCVEngine	Executes exercise-specific biomechanical checks, posture filters, and updates rep count.
connect_player()	BattleManager	Handles private arena code matching, random queue pooling, and AI fallback match initiation.
forward_webrtc_signaling()	BattleManager	Routes SDP offers, SDP answers, and ICE candidate packets between battle participants.
finish_match()	BattleManager	Calculates XP, records wins/losses, handles surrender events, and notifies both players.
estimate_food_calories()	FitnessCoachChatbot	Extracts food keywords and quantities from chat messages and calculates total macros.
initMap()	StravaTracker	Initializes Leaflet map using free OpenStreetMap tiles with GPS polyline rendering.

11. Output & User Interface

The FITBAT system provides a unified, responsive single-page interface consisting of 6 core modules:

- 1. Real-Time Split-Screen Battle Arena:** Displays simultaneous side-by-side feeds showing the user's camera on the left and the opponent's live WebRTC stream on the right. Both viewports feature persistent, non-occluded HUD overlays with live rep counts, combo multiplier gauges, and power bars.
- 2. Automated Pose Tracker Overlay:** Overlays high-contrast skeletal joints and bone segments directly onto the user's live video, accompanied by dynamic feedback prompts (e.g., 'Deep Parallel Squat! Drive up!').
- 3. Outdoor GPS Strava-Style Tracker:** Displays interactive live GPS route mapping on dark-mode OpenStreetMap tiles, live speed/pace gauges, kilometer split times, elevation ascent, and step pedometer metrics.
- 4. Daily Health Monitor & AI Coach:** Renders interactive calorie progress rings, macronutrient breakdown progress cards (Protein, Carbs, Fats), daily hydration cups, and an AI conversational chatbot drawer.
- 5. Dynamic Arena Leaderboards:** Renders global XP leaderboards alongside exercise-specific rankings (Wins, Max Reps in a Single Match, Win Rate) with instant cache-busting refresh buttons.
- 6. Gym Beats Music Player:** Features an interactive multi-genre workout audio player with worldwide iTunes online music search, Web Audio synthesizer, and live spectrum equalizer.

12. Advantages

- 1. Zero Hardware Cost:** Operates directly through standard webcams and mobile browsers without requiring expensive wearable sensors or proprietary fitness watches.
- 2. Biomechanically Accurate Form Validation:** Prevents false reps and injury through multi-point anatomical checks (e.g., verifying horizontal torso for pushups).
- 3. Low Latency Real-Time Multiplayer:** Direct peer-to-peer WebRTC video feeds enable immersive, lag-free competitive battles across different geographic locations.
- 4. Free, Unlimited Mapping:** Employs pure OpenStreetMap tile layers with automatic failover, eliminating expensive Google Maps / Mapbox API key paywalls.
- 5. Unified Holistic Health Solution:** Integrates workout battle gamification, outdoor cardio tracking, and automated dietary calorie monitoring into one platform.

13. Limitations

- 1. Camera Field of View Constraint:** Full-body exercises (e.g., Jumping Jacks, Frog Jumps) require the user to position the camera 6-8 feet away to maintain landmark visibility.
- 2. Lighting & Occlusion Dependency:** Dimly lit workout spaces or loose, baggy clothing can reduce landmark confidence scores below the 0.25 threshold.
- 3. WebRTC NAT Traversal:** While public STUN servers handle standard network configurations, symmetric corporate firewalls may require dedicated TURN relays.
- 4. Geolocation Drift:** Browser GPS accuracy relies on device hardware and satellite reception, which may fluctuate in dense urban high-rise areas.

14. Future Scope

- 1. Native Mobile Applications:** Porting the web application to native Android and iOS using Flutter or React Native with TensorFlow Lite for offline neural inference.
- 2. Wearable Smartwatch Integration:** Synchronizing real-time heart rate zones (BPM) and VO2 max metrics from Apple Watch and Wear OS devices during multiplayer battles.
- 3. Tournament Clan Battles:** Introducing 5v5 team fitness tournaments, bracket elimination leagues, and collegiate fitness championships.
- 4. Computer Vision Barbell / Dumbbell Tracking:** Incorporating object detection (YOLOv8) to track free-weight barbell trajectories and bar velocity for powerlifting.
- 5. Multi-Language Audio Coaching:** Adding localized Hindi, Marathi, and regional voice coaching synthesis for accessibility across rural and urban fitness communities.

15. Conclusion

The **FITBAT** project successfully addresses the prevailing challenges of exercise posture injuries, workout abandonment, and fragmented fitness tracking by uniting **Computer Vision Pose Estimation, Real-Time WebRTC Peer Battling, and AI Dietary Coaching** into an accessible, open-source software ecosystem.

By implementing 3D skeletal joint mathematical algorithms in Python and JavaScript, FITBAT delivers instant, reliable rep counting and form validation with zero proprietary hardware requirements. The platform's real-time multiplayer arena proves that gamification and friendly competition significantly enhance athletic consistency and motivation. FITBAT establishes a robust foundation for next-generation digital sports training and automated biomechanical health monitoring.

*** END OF PROJECT REPORT ***